

Computer Vision Project

Multi-Class Natural Scene Image Classification

Dataset: Intel Image Classification (Kaggle) | Classes: buildings, forest, glacier, mountain, sea, street

Metric	Baseline CNN	Transfer Learning (MobileNetV2)
Test accuracy	83.70%	91.07%
Macro F1-score	0.8400	0.9122
Total parameters	424,006	2,265,670
Training time	812.3 s	406.4 s

1. Project Introduction & Business Problem

This project builds an end-to-end computer vision pipeline that classifies a natural-scene photograph into exactly one of six categories: **buildings**, **forest**, **glacier**, **mountain**, **sea**, **street**. Automatically tagging scene photos at scale is a real need for travel/real-estate platforms, satellite and drone imagery triage, and digital asset management, where manually labeling millions of images is impractical.

Because each image belongs to exactly one of several mutually-exclusive classes, this is framed as a single-label, multi-class classification problem: the model's output layer uses a softmax over six units, trained with categorical cross-entropy, rather than the sigmoid/multi-label setup used when an image can hold several independent tags at once.

The goal is not just high accuracy, but a well-structured, explainable, and reproducible pipeline: every design decision (preprocessing, augmentation, architecture, evaluation) is justified against the actual data, and every result in this report was produced by the accompanying notebook.

2. Dataset Understanding & EDA

Source: Intel Image Classification dataset (Kaggle), downloaded programmatically via *kagglehub*. The dataset ships pre-split into *seg_train*, *seg_test*, and an unlabeled *seg_pred* folder used later for external inference.

Class balance: training counts ranged from 2,191 to 2,512 images per class (imbalance ratio ≈ 1.15) — close enough to balanced that no class-weighting or resampling was needed. **Image quality:** a full scan of all 14,034 training images found zero corrupted/unreadable files, and confirmed every single image is RGB (0 non-RGB found).



Figure 1 — Training images per class.

Visual inspection during EDA flagged **glacier** and **mountain** as sharing overlapping snow/rock textures and color palettes — a strong prior that these two classes would be the most frequently confused by any model trained on raw pixels. This prediction was later confirmed directly in the error analysis (Section 5).

3. Methodology

3.1 Preprocessing & Augmentation

Images were resized to 150×150, forced to 3-channel RGB, and normalized inside the model graph (so normalization is automatically applied at inference time too). Training-only augmentation used horizontal flip, small rotation/zoom/translation, and mild contrast changes — all chosen to reflect realistic camera/framing variation. Vertical flips and heavy hue jitter were deliberately excluded, since color is a genuine class signal here (e.g. blue tones for sea/glacier vs. green for forest) rather than noise to be removed.

3.2 Train / Validation / Test Strategy

The provided `seg_test` folder was held out entirely and used exactly once, for final evaluation only. A validation split was carved out of `seg_train` (15%), giving an overall split close to the suggested 70/15/15. Augmentation was attached only to the training pipeline, so the validation and test sets never see augmented duplicates — directly preventing data leakage between splits.

3.3 Baseline CNN (from scratch)

A four-block Conv2D → BatchNorm → ReLU → MaxPool stack (32→64→128→256 filters), followed by global average pooling and a dense head with Dropout (0.4 / 0.3) for regularization. Trained with Adam, sparse categorical cross-entropy, and EarlyStopping / ReduceLROnPlateau / ModelCheckpoint callbacks.

3.4 Transfer Learning (MobileNetV2)

A second model was built on a MobileNetV2 backbone pretrained on ImageNet — a good fit given its efficiency and strong general-purpose visual features. Training used a two-stage strategy: (1) freeze the base entirely and train a new classification head, then (2) unfreeze only the top ~30 layers of the base and continue training at a much smaller learning rate (1e-5) to gently adapt the highest-level filters to this domain without overwriting well-trained low-level features.

4. Training & Evaluation

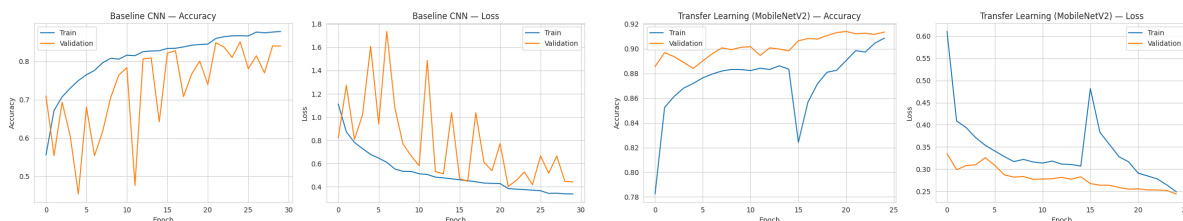


Figure 2 — Training curves: Baseline CNN (left) vs. Transfer Learning (right).

Training accuracy climbed steadily to 87.84% by the final epoch, while validation accuracy reached its best value of 85.13% at epoch 21 (validation loss 0.4042), after which validation performance became more variable. The final train/validation gap (87.84% vs. 84.04%, about 3.8 percentage points) is fairly close, indicating EarlyStopping controlled overfitting reasonably well, consistent with the final test accuracy of 83.70%. The transfer model's curves track even more closely, with fine-tuning (the second phase, at a reduced learning rate) giving a further, more stable improvement.

4.1 Test-Set Results

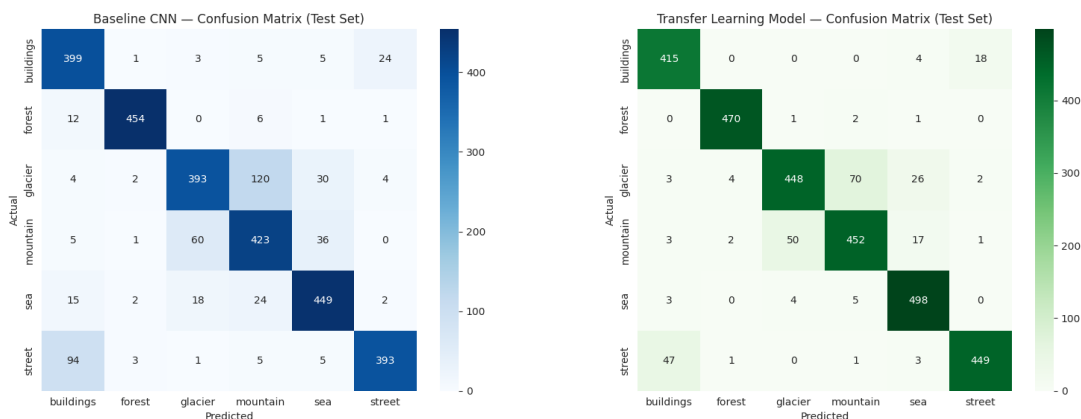


Figure 3 — Confusion matrices: Baseline CNN (left) vs. Transfer Learning (right).

Class	Baseline F1	Transfer F1
buildings	0.83	0.91
forest	0.97	0.99
glacier	0.76	0.85
mountain	0.76	0.86
sea	0.87	0.94
street	0.85	0.92

Table 1 — Per-class F1-score, both models (test set).

glacier and **mountain** have the lowest F1 for the baseline (0.76 each), and **forest** is the easiest for both models (F1 0.97 / 0.99) — consistent with the EDA prediction: forest's distinctive green color/texture makes it easy to separate, while glacier's snow/rock palette overlaps heavily with mountain. By recall specifically, glacier is the single hardest class for the baseline at 71.1%, against forest's 95.8%.

5. Error Analysis

16.3% of test images (489 / 3,000) were misclassified by the baseline CNN. The dominant confusion pattern was **glacier** → **mountain** (120 images, by far the largest single error type), followed by **street** → **buildings** (94), **mountain** → **glacier** (60), and **mountain** → **sea** (36). These recurring pairs are not arbitrary: they reflect genuinely overlapping visual content — snow/rock textures shared between glacier and mountain scenes, and urban geometry shared between street and buildings scenes.

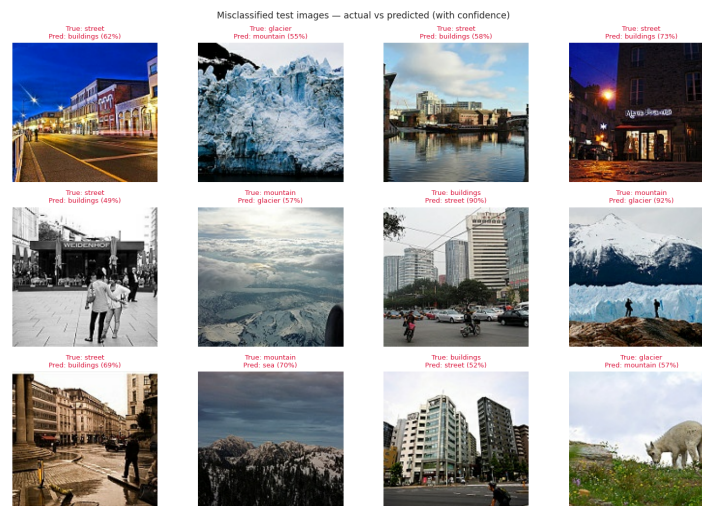


Figure 4 — Sample misclassified test images with ground truth, prediction, and confidence.

6. Explainability — Grad-CAM

Grad-CAM was used to visualize which regions of an image the transfer-learning model relies on for its prediction, by backpropagating the predicted class's gradient into the last convolutional feature map of the MobileNetV2 backbone. This confirms the model is attending to the scene content itself rather than incidental artifacts.

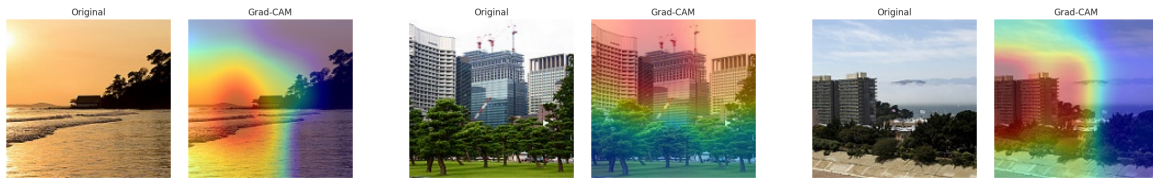


Figure 5 — Grad-CAM heatmaps on three external (seg_pred) images.

7. Conclusion & Future Work

Transfer learning substantially outperformed the from-scratch baseline: test accuracy rose from 83.70% to 91.07%, and macro F1 from 0.8400 to 0.9122, while also training faster in wall-clock time (406s vs. 812s) despite the larger parameter count, since the pretrained backbone converges more efficiently than learning every filter from random initialization. The MobileNetV2 transfer model is therefore selected as the final production model, and is the one deployed in the accompanying prediction function and Streamlit app.

If given more time or data, the following would be prioritized:

- Targeted data collection or augmentation specifically for the glacier/mountain and street/buildings confusion pairs, since generic augmentation does not resolve confusions rooted in genuinely overlapping visual content.
- A deeper fine-tuning sweep and comparison against alternative backbones (EfficientNetB0, ResNet50).
- An ensemble of the baseline and transfer models, or test-time augmentation, to improve robustness on the hardest classes.
- A larger, more geographically/seasonally diverse dataset to reduce dataset-specific texture correlations exposed by the error analysis.

References

- Kaggle dataset: kaggle.com/datasets/puneet6060/intel-image-classification
- Keras Applications: keras.io/api/applications/
- TensorFlow image classification guide: tensorflow.org/tutorials/images/classification
- TensorFlow transfer learning guide: tensorflow.org/tutorials/images/transfer_learning